

FinVest Part A and Part B Detailed Explanation

Prepared from the latest project state in `D:\FinVest\backend`. This version focuses only on Part A and Part B, but in much more depth, so teammates can study implementation logic properly for review and evaluator questions.

Reading note: This document intentionally stays focused on exactly two things only:

1. **Part A:** user story wise implementation explanation
2. **Part B:** module wise implementation explanation

No extra architecture chapters are inserted in between. Instead, the technical depth is pushed inside every user story and every module so the team can answer “how exactly did you implement it?” rather than only “what does it do?”.

Part A: User Story Wise Detailed Explanation

US-1: As a user, I want to register and log in so I can access my finance dashboard

What this story means functionally

This story is the entry gate into the whole system. Until a user is authenticated, none of the personal finance modules such as dashboard, transactions, budgets, goals, reports, or account settings should be accessible. So this story is not only about creating a user row in the database; it is about establishing identity, enforcing uniqueness, protecting credentials, creating a session, and then routing the user into the correct role-based interface.

Where it is implemented

- `backend/app/routes/auth_routes.py`
- `backend/app/templates/auth/auth.html`
- `backend/app/static/css/auth.css`
- `backend/app/models/user_model.py`
- `backend/app/services/active_session_service.py`
- `backend/app/services/admin_service.py`

Backend registration flow in detail

1. The signup form sends `username`, `email`, `password`, `confirm_password`, and all recovery-question answers to `/register`.
2. The backend first checks for missing values. This prevents partial registration state.
3. The username is cleaned with `strip()` and checked against `USERNAME_PATTERN`, which allows only letters, numbers, underscore, and dot.
4. The email is normalized to lowercase before lookup. This avoids duplicate identities such as uppercase and lowercase versions of the same mail address.
5. The password is checked against `PASSWORD_PATTERN`. The pattern requires uppercase, lowercase, numeric digit, special symbol, and minimum length. This is stronger than a simple minimum-length-only rule.
6. The backend verifies that `password == confirm_password` so the user does not accidentally create a broken login.
7. The system checks whether the email already exists in the `users` table. If yes, registration is rejected.
8. The code rejects the reserved admin email from public signup. This keeps system-admin identity separate from general user registration.
9. The password is hashed using Werkzeug's `generate_password_hash()`.
10. The recovery answers are processed by `_security_payload_from_form()`, which collects each answer, normalizes it, and hashes it before storing the JSON payload in `users.security_answers`.
11. The user is inserted into MySQL through SQLAlchemy.
12. Immediately after user creation, the backend creates required dependent records:
 - a `user_settings` row
 - default categories
 - a default account

13. That means the user can log in and use the dashboard directly without a setup wizard to create the first account or first category manually.

Backend login flow in detail

1. The login form posts email and password to `/login`.
2. Email is normalized to lowercase.
3. The code keeps login-attempt metadata in Flask session using keys like `login_attempts` and `last_attempt_time`.
4. If too many attempts happen within the cooldown window, the backend blocks login temporarily and flashes a message.
5. If the user email is not found, attempts are increased and the login is rejected.
6. If the password hash check fails, attempts are increased and the login is rejected.
7. If the account exists but `is_active` is false, login is denied because the admin has blocked the user.
8. If login succeeds, the backend saves identity details in Flask session:
 - `user_id`
 - `username`
 - `email`
 - `role`
 - `session_token`
9. The session token is also recorded in `active_sessions` through `start_active_session()`.
10. An activity log row is created using `log_activity("login", ...)`.
11. The route redirects admins to `dashboard.admin_dashboard` and normal users to `dashboard.home`.

Frontend behavior

The auth page is not two separate HTML files. The same template contains both sign-in and sign-up containers and toggles them through CSS class switching. That means the UI remains visually consistent and the route logic stays centralized.

What teammates should say if asked tricky questions

- **How is password security handled?** Passwords are hashed with Werkzeug before storage.
- **How is account recovery implemented?** The system stores hashed answers for recovery questions and verifies those during password reset.
- **How does the app know who is currently logged in?** Flask session stores the current token and identity, and a corresponding record is stored in `active_sessions`.

US-2: As a user, I want to set currency, locale, and month-start so reports are meaningful

Functional meaning

This story is about personalization of financial display and planning behavior. Without settings, every user would see the same currency and same default assumptions, which is not ideal for a finance app. This story also becomes more important in the updated version because monthly salary automation depends on settings data too.

Where it is implemented

- `backend/app/routes/dashboard_routes.py`
- `backend/app/models/user_settings_model.py`
- `backend/app/templates/dashboard/account.html`
- `backend/app/services/currency_service.py`
- `backend/app/services/i18n_service.py`

Technical flow

1. When the user opens `/account`, the backend loads the user row and the settings row using `get_or_create_user_settings()`.
2. If settings do not exist, they are created automatically.
3. The account page displays fields for currency, locale, language, month-start day, and monthly salary.
4. When the form is submitted to `/account/settings`, the route validates and stores:
 - currency code
 - locale string
 - normalized language code
 - month-start day clamped between 1 and 28

- monthly salary value as Decimal
5. The route logs the change using `log_activity("settings_updated", ...)`.
 6. The route also triggers `process_monthly_salary()`, which means settings update is connected to automation logic, not only UI preference storage.

Why this story matters technically

The settings row influences multiple systems:

- dashboard and report display currency
- layout-wide context injection
- language normalization
- automation engine for salary posting

So if asked "is settings only a profile feature?", the right answer is no. It feeds both presentation logic and financial automation logic.

US-3: As a user, I want to add income entries so monthly inflows are tracked

Functional meaning

This story ensures that positive cash flow is explicitly stored and then reflected in both account balances and analytical summaries such as monthly income, net total, and income-source analysis.

Where it is implemented

- `backend/app/routes/transaction_routes.py`
- `backend/app/models/transaction_model.py`
- `backend/app/services/transaction_service.py`
- `backend/app/templates/dashboard/transactions.html`

Detailed flow

1. The user chooses transaction type `income`, account, category, amount, date, and description.
2. The route ensures that the referenced account and category belong to the logged-in user.
3. Amount is converted into `Decimal` to avoid floating-point precision issues in finance logic.
4. The transaction is inserted into the `transactions` table.
5. The route calls the transaction service to apply a positive effect to the linked account balance.
6. The action is logged for audit purposes.

Why this design is important

The app does not wait until dashboard rendering to infer account balance from transaction history. Instead, it maintains account balances as a fast summary, updating them during every transaction write. That gives faster reads and immediate UI response.

US-4: As a user, I want to add expenses and categorize them for analysis

Functional meaning

Expense entries are the core of spending analysis. Without properly categorized expenses, the dashboard, reports, and budgets would all become weak.

Technical implementation

The same transaction route handles both income and expense entries. The difference is in the type field and the financial side effect:

- `income` adds to balance
- `expense` subtracts from balance

Expenses must point to a valid expense category belonging to that user. Once saved, they become part of:

- monthly expense KPI
- pie chart category distribution
- budget comparison

- report category analysis
- top-spending-category detection

Evaluator angle

If asked “how do you know the user spent more on Rent than Clothes?”, the answer is that all expense transactions are grouped by `category.name` in the dashboard and report aggregation code.

US-5: As a user, I want to set category budgets and get threshold alerts

Functional meaning

This story turns passive expense recording into active spending control. The user does not only see what happened, but also whether actual spending is moving outside the planned limit.

Where it is implemented

- `backend/app/routes/budget_routes.py`
- `backend/app/models/budget_model.py`
- `backend/app/services/notification_service.py`
- `backend/app/templates/dashboard/budgets.html`
- `backend/app/templates/dashboard/home.html`

Detailed flow

1. The user submits category, limit amount, month, and year.
2. If a budget for the same category-month-year already exists, the route updates it instead of creating a duplicate row.
3. While rendering budgets or dashboard home, the code computes current-month expense totals by category.
4. For each saved budget, the route calculates:
 - spent amount
 - remaining amount
 - usage percentage
5. The notification service compares the percentage with thresholds:
 - 80% creates a budget warning
 - 100% or more creates a budget exceeded alert

Technical strength

The budget row is not stored with a “current percent used” column. Instead, actual spend is derived from current transactions, which keeps budget state consistent with real data.

US-6: As a user, I want to create savings goals and track progress

Functional meaning

This story supports planning future savings targets such as an emergency fund, travel plan, laptop purchase, or any user-defined financial goal.

Where it is implemented

- `backend/app/routes/goal_routes.py`
- `backend/app/models/goal_model.py`
- `backend/app/templates/dashboard/goals.html`

Detailed flow

1. The goal create route accepts goal name, target amount, current amount, deadline, and status.
2. The route inserts the row and logs creation.
3. The goal update route lets the user modify structural fields.
4. The dedicated add-amount route increases the current saved progress without forcing the user to resubmit the entire goal definition.
5. Dashboard home reads the top goal and computes a percentage:
 - `current_amount / target_amount * 100`
 - clamped between 0 and 100 for safe rendering

6. The notification service checks whether a goal deadline is close and creates deadline alerts.

Why goals are stored separately instead of deriving from transactions

Not every savings goal maps perfectly to one transaction category. A user may contribute irregularly from many sources. So storing progress directly as goal state is a simpler and more flexible approach for the project.

US-7: As a user, I want charts and KPIs to understand finances quickly

Functional meaning

This story is about transforming raw data into high-signal information. Instead of showing only transaction tables, the app summarizes key indicators so the user can understand current financial position quickly.

Where it is implemented

- `backend/app/routes/dashboard_routes.py`
- `backend/app/templates/dashboard/home.html`
- `backend/app/static/css/dashboard.css`

Detailed data pipeline

1. The home route calls `_build_dashboard_data(user_id)`.
2. This function ensures settings, categories, and default account exist.
3. It loads transactions, accounts, goals, and current-month budgets.
4. It computes total balance from accounts.
5. It computes current month income and expense from transactions.
6. It builds monthly grouped data for the last four months.
7. It derives average monthly expense from recent non-zero months.
8. It groups current-month expenses by category to create chart data.
9. It creates budget comparison rows and goal summary rows.

Technical rendering approach

The app does not depend entirely on client-side JavaScript charting libraries for the main dashboard view. Instead, it precomputes widths and pie chart style strings server-side. This makes the UI predictable and keeps the logic near the financial data.

US-8: As a user, I want to search and filter transactions effectively

Functional meaning

The user should not be forced to manually scan a large transaction list. This story enables historical exploration by narrowing results according to meaningful business filters.

Detailed implementation

1. The transactions page accepts optional query parameters:
 - `type`
 - `category_id`
 - `start_date`
 - `end_date`
 - `min_amount`
 - `max_amount`
2. The route starts from `Transaction.query.filter_by(user_id=user_id)`.
3. Each filter is parsed and conditionally added to the SQLAlchemy query.
4. Date strings are converted into `datetime.date` objects before comparison.
5. Amounts are converted into `Decimal` before comparison.
6. The filtered query is ordered by transaction date descending and then id descending.

Technical importance

Filtering happens in the database query, not after loading everything into memory on the browser side. That is the correct pattern for controlled and scalable data retrieval.

US-9: As a user, I want to import transactions from a CSV template

Functional meaning

This story reduces manual data entry effort and allows external financial data to be pulled into the system.

Where it is implemented

- `backend/app/routes/transaction_routes.py`
- `backend/app/static/samples/dummy_transactions.csv`

Detailed flow

1. User uploads a CSV file through the transaction module.
2. The route checks whether the file is present and whether headers exist.
3. Required headers are validated: date, type, category, amount.
4. Each row is cleaned through helper functions so field names and values are normalized.
5. Dates are parsed using fixed format logic.
6. Amounts are parsed as Decimal values.
7. The route attempts to find or create the category for the imported row.
8. The route attempts to find a matching account or fallback account.
9. The route checks for duplicates using a multi-field identity match:
 - user
 - account
 - category
 - type
 - amount
 - date
 - description
10. Non-duplicate rows are inserted and their account balance effect is applied.
11. Duplicate rows are counted and skipped.

What teammates should mention

The import feature is not just file upload. It contains validation, normalization, deduplication, category/account resolution, transaction insertion, and balance synchronization.

US-10: As a platform owner, I need secure access and auditable changes

Functional meaning

This story is about protecting sensitive operations and leaving a trace for important actions in the system.

Where it is implemented

- `backend/app/utils/decorators.py`
- `backend/app/utils/activity_logger.py`
- `backend/app/models/activity_log_model.py`
- `backend/app/models/active_session_model.py`
- `backend/app/services/active_session_service.py`

Detailed behavior

1. Protected routes use decorators such as `login_required` and `admin_required`.
2. User identity is stored in Flask session after login.
3. Important actions create audit entries, for example:
 - login
 - logout
 - transaction create/update/delete

- budget create/update/delete
 - goal update
 - account changes
 - admin block/unblock/delete actions
4. Active session rows are touched continuously during authenticated requests so the system can differentiate currently active users from old historical logins.

Evaluator angle

If someone asks “how is your admin panel not open to everyone?”, the answer is that admin routes are explicitly protected by the admin decorator and role checks.

US-11: As a user, I want a welcome or onboarding guide

Functional meaning

This story helps first-time users understand what to do next and reduces confusion when entering a multi-module application.

Implementation details

The project implements onboarding in two layers:

- Public landing page before login through `auth.onboarding()` and `onboarding.html`.
- Quick guide inside the logged-in dashboard through `show_guide` and `dismiss_guide` session logic.

The dashboard guide is not hardcoded permanently visible. It can be hidden and shown again, which gives a more realistic user experience.

US-12: As a user, I want to view monthly income vs expense comparison

Technical meaning

This story depends on historical aggregation over time, not just simple totals.

Detailed implementation

1. The dashboard builds a four-month rolling window.
2. Transactions are grouped by year-month key.
3. Income and expense are accumulated separately.
4. The largest value across the series becomes the normalization divisor.
5. Each month receives an `income_width` and `expense_width` percentage.
6. The template renders these widths as bars so that higher income/expense visually occupies more horizontal space.

Why this matters

This is a simple but technically sound server-driven chart pattern that avoids duplicating finance logic in JavaScript.

US-13: As a user, I want category-wise spending analysis

Detailed implementation

The dashboard filters current-month expense transactions and groups them by category. It sorts categories by descending spend and builds:

- a pie chart style string using `conic-gradient`
- a legend list containing name, amount, percent, and color
- top category information for special highlighting

The reports page also provides category analysis in a more analytic list-oriented form. So this story is implemented in both dashboard and reports, but with different presentation goals.

US-14: As a user, I want to view my average monthly spending

Detailed implementation

The dashboard uses recent grouped monthly expense totals. Months with zero expense are filtered out before averaging. This avoids dragging the average down artificially when the dataset contains inactive months. The result is displayed as its own KPI card on the dashboard.

US-15: As an admin, I want to view account activity history

Detailed implementation

1. The admin dashboard loads activity logs primarily from MongoDB.
2. If Mongo logs are unavailable, it falls back to SQL-based activity logs.
3. The app converts both source shapes into a normalized row structure with:
 - user_name
 - action
 - details
 - created_at_label
 - created_on (normalized date key)
4. The normalized rows are reused for:
 - activity feed display
 - today login count
 - today logout count
 - last 7 days activity chart

Why this is a good implementation

It gives the admin both raw event visibility and derived metrics from the same source data.

US-16: As a user, I want to view my budget vs actual spending

This is implemented by computing current-month expense totals by category and matching them against stored budgets for the same month and year. The app derives percent usage and renders visual status accordingly. The logic is visible in both the dedicated budget module and the summarized dashboard section, which makes this story highly visible to the evaluator.

US-17: As a user, I want to edit or delete my transactions

Detailed implementation

The important technical point is not the form update itself, but the balance correction mechanism:

1. When editing, the system finds the old transaction and old account.
2. It reverses the old transaction's financial effect from the old account.
3. It updates the transaction fields.
4. It applies the new effect to the target account.
5. It commits and logs the update.

Delete follows the same principle but stops after reversal and deletion. This logic is exactly what keeps the total balance card and account balances correct over time.

US-18: As a user, I want to add notes or descriptions to my transactions

This is implemented through the `description` field on the transaction model. The add form, edit form, CSV import, dashboard recent-transactions display, CSV export, JSON backup, and reports all preserve this field. That means descriptions are first-class data in the system, not temporary UI-only text.

US-19: As a user, I want to mark my notifications as read

The notifications table has an `is_read` boolean field. The notifications blueprint exposes routes to mark a single notification as read or mark all as read. The unread count shown in the header bell is calculated centrally through the app context processor, so every page sees the same number

without repeating query logic in each template.

US-20: As a user, I want to view my transaction history for a selected date range

This is implemented directly inside the transaction list route. The route accepts start and end dates, converts them into real date objects, and constrains the query using greater-than-or-equal and less-than-or-equal filters on `Transaction.date`. The result can then be explored on the page or exported through the reports module.

Part B: Module Wise Detailed Explanation

Module 1: Authentication and Authorization Module

Purpose of the module

This module establishes user identity, protects access to the rest of the application, separates normal users from admin users, and provides controlled account recovery.

Major technical responsibilities

- registration validation
- email uniqueness enforcement
- password hashing
- role-aware login redirect
- login-attempt throttling
- security-question based password reset
- active-session creation and cleanup
- admin seeding

How the parts connect

The route layer handles HTTP input and error messaging. The user model stores identity and security answer payload. The active-session service tracks who is active. The admin service ensures that the reserved admin account exists and has the right role. The decorators then use session state and role information to enforce access rules throughout the project.

What evaluators may ask

They may ask why recovery is question-based instead of email-based. The safest answer is that the current implementation chose a database-only recovery method to avoid mail-service complexity while still protecting user accounts with hashed recovery answers.

Module 2: Transaction Management Module

Purpose of the module

Record and maintain all money movement in the system and keep account balances synchronized with those records.

Main technical components

- `transaction_model.py`
- `transaction_routes.py`
- `transaction_service.py`
- `transactions.html`

Core design decision

The system stores both transaction history and current account balance. That means balance is not recalculated every time by summing all transactions. Instead, balance changes are applied during create, update, delete, and import operations.

Sub-features inside this module

- manual income/expense creation
- description storage
- date/category/type/amount filtering
- transaction edit
- transaction delete
- CSV import with deduplication

Why this module is technically important

Almost every other module depends on transaction integrity: budgets compare against transactions, dashboard KPIs summarize them, reports aggregate them, notifications inspect them, and account balances are driven by them.

Module 3: Category Management Module

Purpose

Provide user-specific classification for both income and expense records.

Design behavior

- Default categories are seeded automatically for every user.
- Categories are stored per user, not globally.
- Each category has a `type` of either income or expense.

Why this design matters

Per-user categories avoid conflicts and let the user personalize their finance classification while still starting with a sensible base set. Since budgets and reports depend heavily on category grouping, keeping categories user-scoped is a clean design choice.

Module 4: Dashboard Module

Purpose

Serve as the main financial summary screen where the user can understand balance, monthly flows, category distribution, recent activity, goals, and budgets at a glance.

Internal structure

The dashboard route does not scatter its calculations across many templates. Instead, it centralizes most home-screen analytics in `_build_dashboard_data()`. This helper composes data from accounts, transactions, budgets, goals, and settings into one structured payload.

Data shown

- total balance
- monthly income
- monthly expense
- average monthly spending
- monthly income vs expense chart
- category-wise spending pie chart
- top spending category
- budget vs actual section
- recent transactions
- top saving goal
- quick guide panel

Why this is good technically

By building all this on the server, the project keeps financial logic in Python where validation and aggregation are easier to reason about. The template layer stays focused on visual display of already-processed results.

Module 5: Budget Planning Module

Purpose

Help users define spending limits by category and time period, compare those limits against actual behavior, and also manage recurring expenses that affect future budget usage.

Core sub-features

- create/update monthly budgets
- show current spend vs limit

- show remaining budget
- generate threshold notifications
- manage recurring expense definitions

Why recurring expenses are here

Recurring expense definitions logically belong in the spending-planning area because they represent expected repeat outflows. The user configures these from the budget page, and then the automation service later turns them into real transactions.

Technical flow for recurring expenses

1. User chooses account, category, amount, frequency, start date, and description.
2. The system inserts a `recurring_transactions` row.
3. On future requests, due rows are turned into actual transaction entries.
4. Those generated entries affect account balance and all downstream analytics just like manually entered expenses.

Module 6: Reports and Analytics Module

Purpose

Provide deeper analysis and export capability beyond the quick dashboard summary.

Main components

- `report_routes.py`
- `report_service.py`
- `reports.html`

What this module computes

- total income
- total expense
- net
- average monthly spending
- savings rate
- top expense category
- income source contributions
- monthly income/expense/net rows
- category chart rows
- trend statements
- transaction count

Mongo snapshot behavior

When report generation runs with snapshot saving enabled, the service builds a snapshot payload, hashes it using SHA-256 to create a fingerprint, and stores it in MongoDB only if an identical fingerprint does not already exist. This avoids duplicate snapshot clutter from repeated refreshes.

Why this module is technically strong

It shows real aggregation logic, report persistence, export generation, and integration of both SQL and Mongo data sources.

Module 7: Reminder and Notifications Module

Purpose

Surface actionable events to the user without requiring them to manually inspect every module continuously.

Notification sources

- welcome notification
- budget threshold alerts
- budget exceeded alerts

- goal deadline alerts

How duplicates are prevented

The notification service stores a `reference_key` for each logical event. Before inserting, it checks if the same user already has that event key. If yes, the notification is not inserted again. This is the main anti-duplication mechanism.

User interaction

The user sees unread count in the bell icon and can mark one or all notifications as read. This makes notification state persistent and interactive rather than just ephemeral popups.

Module 8: Data Export and Backup Module

Purpose

Allow users to take structured data out of the system for spreadsheet work, backup, or review.

Two main outputs

- **CSV export:** includes analytics summary, trends, chart data, and transaction rows.
- **JSON backup:** includes user, settings, accounts, categories, goals, recurring transactions, transactions, and analytics.

Why this is more than a simple dump

The JSON output is not just table rows. It is shaped into a meaningful structured payload with nested sections, which makes it easier for restore workflows or external inspection later.

Module 9: Saving and Goals Module

Purpose

Track long-term savings objectives separate from daily transaction mechanics.

Technical choices

- Goals are stored independently of transaction category totals.
- Progress is editable and incrementable through a dedicated add-amount route.
- The dashboard reads and displays the top goal summary.
- Deadline awareness is integrated through notifications.

Why this approach is practical

It supports realistic user behavior where goal progress may be updated manually based on many financial actions rather than linked to one strict transaction class.

Module 10: Account Management Module

Purpose

Let users manage profile identity, password, settings, and financial accounts used by transactions.

Main responsibilities

- update display name
- change password after current-password verification
- update settings such as currency and salary
- add accounts like Cash, Bank, UPI
- delete accounts

Why this module is critical

Accounts are the containers that hold balances, and transactions always point into accounts. So account management is not a side feature; it is a foundational data domain of the entire finance system.

Module 11: Admin and System Module

Purpose

Provide system-level visibility and control over users, activity, transactions, session state, and performance metrics.

Main admin features implemented

- view all users
- view blocked/active status
- block or unblock users
- delete users
- view system transactions
- filter transactions by selected user
- view activity log feed
- view daily login/signup trends
- view active sessions
- view DB response and request timing

Performance sub-module

The synced code added a dedicated admin performance screen. This page measures active logins through session tracking, database response through a lightweight query timer, last request duration through app-level response timing, and log volume through Mongo or SQL counts.

Why this is technically valuable

This module shows that the project is not only user-facing finance CRUD. It also has admin observability, operational monitoring, and action traceability.